

Classical & Quantum Signature Data

This document formalizes the development of **two strictly separated signature schemes**:

- **C-SQD**: classical-only signatures for bitstrings.
- **Q-SQD**: quantum signatures for quantum states/circuits, designed to be compatible with quantum mechanics and robust to sampling noise.

The split is deliberate to avoid the conceptual conflation in the original PDF (e.g., treating a sign function as “superposition probability,” and treating $2S + 1 = 4$ as a “prime signature”).

1. Design goals

Non-negotiables

- **No classical/quantum conflation**: C-SQD operates only on bitstrings; Q-SQD operates on quantum states/circuits.
- **Auditability**: optional “witness” representations exist but are not required for storage.
- **Noise realism (Q-SQD)**: signatures must include uncertainty reporting and instability flags.
- **JSON I/O**: all outputs are JSON-serializable.

What SQD is / is not

- SQD is a **signature / fingerprint**, not necessarily an injective encoding (Q-SQD).
 - C-SQD is injective for fixed n (by unique factorization), but the stored representation uses sparse indices.
-

2. C-SQD v1.0 (Classical)

2.1 Input

- Bitstring $b \in \{0, 1\}^n$, provided as:
- string: `"1011"`, or
- list/tuple: `[1, 0, 1, 1]`.

2.2 Output JSON schema

```
{
  "version": "C-SQD/1.0",
  "n": 4,
  "e": [0, 2, 3],
  "M": {"mode": "hamming", "value": 4},
}
```

```

    "C": "<hex>",
    "P_witness": "<optional decimal big-int>"
}

```

2.3 Definitions

Microstate (stored)

Let $e \subseteq \{0, \dots, n-1\}$ be indices where $b_i = 1$.

- Stored microstate: $e = [i \mid b[i]==1]$.

Optional prime witness (audit)

Let p_i be the $(i+1)$ -th prime (2,3,5,7,...).

$$P(b) = \prod_{i=0}^{n-1} p_i^{b_i}.$$

Note: $P(b)$ can be huge; it's an *optional witness*, not recommended as the stored form.

Multiplicity (macro descriptor)

- **Hamming mode (default):**

- $k = \sum b_i = |e|$

- $M = \binom{n}{k}$

- **Orbit mode (optional):** for a declared group action G on indices,

$$M = |G \cdot b|.$$

Default example: **cyclic rotations**.

Checksum

$$C = \text{HASH}(\text{canonical}(b \text{ or } e)).$$

Recommended:

- BLAKE3 truncated (fast) if available
- else SHA-256 truncated.

2.4 Fixing the PDF example

For `[1, 0, 1, 1]`:

- `e=[0, 2, 3]`
- `M=binom(4, 3)=4` (multiplicity)
- `P_witness=2*5*7=70` (invertible micro-ID)

This preserves “4” as a valid multiplicity and avoids the incorrect claim that it is prime.

3. Q-SQD v1.0 (Quantum)

3.1 Input

- A quantum state **via a circuit** (reference implementation uses Qiskit Aer simulation).

3.2 Feature family (fixed default)

- All **Pauli** strings of weight ≤ 2 on n qubits.
- Each feature is an expectation value: $f_k(\rho) = \mathrm{Tr}(\rho O_k) \in [-1, 1]$.

Purities and entanglement witnesses can be added; the reference implementation focuses on Pauli features + uncertainty reporting.

3.3 Estimation from shots

For each Pauli observable O_k , measure in the appropriate basis, gather counts, estimate:

- $\hat{f}_k$ (sample expectation)
- standard error `se_k` from a Bernoulli model on ± 1 outcomes.

3.4 Quantization (with guard band)

Choose resolution B and guard parameter λ (default $\lambda = 2$).

$$q_k = \left\lfloor B \hat{f}_k \right\rfloor$$

Flag feature k as **unstable** if too close to the bin center relative to uncertainty:

$$\left| \hat{f}_k - \frac{q_k}{B} \right| < \lambda \mathrm{se}_k$$

3.5 Output JSON schema

```
{  
  "version": "Q-SQD/1.0",
```

```

    "n": 3,
    "featureset": {"family": "pauli", "max_weight": 2},
    "q": {"k17": 3, "k29": -1},
    "h": "<sha256 hex>",
    "eta": {
      "N": 2000,
      "B": 50,
      "lambda": 2.0,
      "se": {"k17": 0.04, "k29": 0.05},
      "unstable": ["k29"],
      "flip_rate_pred": 0.03
    },
    "Gamma_witness": "<optional decimal big-int>"
  }

```

3.6 Witness + digest

- Digest: $h = \text{SHA-256}(\text{canonical}(q))$.
- Optional witness:

$$\Gamma = \prod_k p_k^{g(q_k)}$$

for injective $g : \mathbb{Z} \rightarrow \mathbb{N}$.

4. Expected outcomes (falsifiable)

4.1 C-SQD

- **Injective microstate witness** (fixed n): no collisions.
- **Checksum detection**: random flips almost surely change C .

4.2 Q-SQD

- **Stability**: repeated-run disagreement of q decreases as shots N increase.
- **Separation**: benchmark states (product vs Bell/GHZ/W) yield distinguishable q vectors at reasonable (B, N) .
- **Calibration**: observed flip rate should not substantially exceed $\text{eta.flip_rate_pred}$.

5. Fastest validation path (pass/fail)

5.1 C-SQD (minutes)

- Sample many bitstrings, compute C-SQD.
- Flip bits at BER = 1% and 10%.

- **Pass** if:
 - decoding from `e` returns original bitstring,
 - checksum mismatch rate ≈ 1 for flips.

5.2 Q-SQD simulation (hours)

Benchmarks on $n = 3$: $|000\rangle$, Bell $\otimes|0\rangle$, GHZ, W, depolarized variants.

For each state:

- Compute Q-SQD at $N \in \{100, 500, 2000, 10000\}$, $B \in \{10, 50, 100\}$.
- Repeat K times to measure stability.

Pass if:

- collisions among benchmark families $< 1\text{--}5\%$ at some reasonable (B, N) ,
- stability improves with N ,
- calibration holds: observed flip rate $\lesssim$ predicted.

6. Reference implementation (Python + Qiskit Aer)

The reference code lives at:

- `sqd_reference.py`

It implements:

- `c_sqd(b, mode='hamming'|'orbit', witness=False)`
- `q_sqd(circuit, shots, B, lambda_guard, witness=False)`

6.1 Code snippet: C-SQD usage

```
from sqd_reference import c_sqd

out = c_sqd([1,0,1,1], mode='hamming', witness=True)
print(out) # JSON-serializable dict
```

6.2 Code snippet: Q-SQD usage (Aer)

```
from qiskit import QuantumCircuit
from sqd_reference import q_sqd

qc = QuantumCircuit(3)
qc.h(0)
```

```
qc.cx(0, 1)
qc.cx(1, 2)  # GHZ

out = q_sqd(qc, shots=2000, B=50, lambda_guard=2.0, witness=False)
print(out)
```

6.3 CLI idea (optional)

A minimal CLI can be added to read JSON on stdin and write JSON on stdout for both branches.

7. Implementation notes

- Big-int witnesses (`P_witness`, `Gamma_witness`) should be treated as **optional audit artifacts**, not required for storage.
 - Canonical serialization matters for stable digests. Use deterministic ordering for `q` keys and `e` indices.
 - Q-SQD instability reporting is a core safety feature: it prevents over-trusting quantized signatures near bin boundaries.
-

One simple injective map $g : \mathbb{Z} \rightarrow \mathbb{N}$:

$$g(z) = 2|z| + \mathbf{1}_{\{z < 0\}}.$$

This encodes sign in parity and magnitude in the quotient.

Appendix B — Pauli measurement basis changes

To measure expectation of a Pauli string:

- For each qubit:
- Z: measure in computational basis.
- X: apply H then measure.
- Y: apply $S^\dagger$ then H then measure.

Then map each observed bitstring to ± 1 outcome via parity on the measured qubits.

References

1. Gödel, K. *Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I*. **Monatshefte für Mathematik und Physik**, 38, 173–198 (1931).

2. Origin of unique arithmetization ideas underlying prime-factor encodings.
3. Hardy, G. H. & Wright, E. M. *An Introduction to the Theory of Numbers*. Oxford University Press.
4. Fundamental theorem of arithmetic and prime factorization.
5. Nielsen, M. A. & Chuang, I. L. *Quantum Computation and Quantum Information*. Cambridge University Press (2000).
6. Density matrices, Pauli observables, and expectation values.
7. Aaronson, S. *Shadow Tomography of Quantum States*. **SIAM Journal on Computing** 49(5), 2019.
8. Quantum state fingerprinting and observable-based summaries.
9. Huang, H.-Y., Kueng, R., & Preskill, J. *Predicting many properties of a quantum system from very few measurements*. **Nature Physics** 16, 1050–1057 (2020).
10. Classical shadows and scalable estimation of Pauli observables.
11. Hoeffding, W. *Probability inequalities for sums of bounded random variables*. **Journal of the American Statistical Association** 58(301), 13–30 (1963).
12. Concentration bounds motivating shot-noise error estimates.